Institución Universitaria Pascual Bravo

Ingeniería de Software

SolarBI Pascual

Power BI y Grafana aplicados al análisis de datos solares

Santiago Velásquez Serna

1038414022

Inteligencia de Negocios · Grupo 50 · Semestre 2026-II

Docente: Ramiro Grisales Montoya

Medellín

9 de octubre de 2026

Contenido

Parte A. Consulta 3

Bloque 1. Power BI y Grafana 3

D1. ¿Qué es Grafana? 3

D2. Diferencias entre Power BI y Grafana 3

D3. Tipos de decisiones 3

D4. Import y DirectQuery 3

D5. Variables y filtros 4

Bloque 2. Gobernanza de datos 4

G1. Gobernanza, gestión y calidad 4

G2. Responsables de los datos 4

G3. Acuerdo sobre los datos 4

G4. Una sola fuente de datos 4

G5. Protección de datos personales 5

Bloque 3. Automatización del proceso ETL 5

E1. ETL, ELT y Power Query 5

E2. Carga incremental e idempotencia 5

E3. Formas de automatizar el proceso 5

E4. Actualización incremental en Power BI 5

Bloque 4. Internet de las Cosas 6

I1. Recorrido de los datos 6

I2. MQTT y entrega de mensajes 6

I3. Protocolos industriales 6

I4. Datos por lotes y en tiempo real 6

Bloque 5. Control de versiones 7

V1. Git, GitHub y archivos del proyecto 7

Parte B. Práctica guiada 8

Paso 1. Capa Bronze 8

Paso 2. Limpieza y capa Silver 8

Paso 3. Capa Gold e idempotencia 9

Paso 4. Informe en Power BI 10
Paso 5. Dashboard en Grafana 10
Paso 6. Reflexión 12
Parte C. Repositorio y verificación 13
Referencias 14

Parte A. Consulta

Bloque 1. Power BI y Grafana

D1. ¿Qué es Grafana?

Grafana es una herramienta para ver datos en gráficos y tableros. Primero se crea una conexión con la fuente de datos. Después, cada panel hace una consulta y muestra un resultado. Varios paneles forman un dashboard. Las variables permiten cambiar filtros y las alertas avisan cuando ocurre una condición definida. Grafana OSS se instala en un equipo propio, mientras que Grafana Cloud funciona como un servicio en internet. En este proyecto usé Grafana OSS en mi computador, conectado a PostgreSQL. [1]

D2. Diferencias entre Power BI y Grafana

Power BI sirve más para analizar el negocio y crear cálculos. Grafana sirve más para revisar datos que cambian con frecuencia. En este proyecto usé Power BI para mostrar energía, rendimiento y ahorro. Usé Grafana para revisar las lecturas y la calidad de los datos.

Criterio	Power BI	Grafana
Propósito	Análisis del negocio	Seguimiento de datos
Usuario	Analista o directivo	Operador o técnico
Datos	Import o DirectQuery	Consulta directa a la fuente
Semántica	Modelo y fórmulas DAX	Consultas SQL
Actualización	Actualización del modelo	Actualización automática
Alertas	Servicio de Power BI	Reglas de alerta
Costo	Desktop + licencias de servicio	Local o en la nube
Acceso	Espacios y permisos	Roles y carpetas

D3. Tipos de decisiones

Las decisiones operativas resuelven lo que está pasando en el momento. Las decisiones analíticas ayudan a entender tendencias y causas. Las estratégicas sirven para revisar metas generales. En Grafana mostraría alertas, disponibilidad y porcentaje de datos válidos. En Power BI mostraría energía, rendimiento y ahorro. Así, Grafana ayuda al operador y Power BI ayuda a quien necesita un resumen para decidir.

D4. Import y DirectQuery

Con Import, Power BI guarda una copia de los datos y responde rápido, pero esa copia debe actualizarse. Con DirectQuery, Power BI consulta PostgreSQL cada vez que el usuario interactúa con el informe. Para los tres días de datos de SolarBI usé Import porque el volumen es pequeño y permite trabajar bien con DAX. Una actualización diaria es suficiente. Grafana sí puede consultar PostgreSQL cada cinco minutos. Si el proyecto creciera mucho, podría evaluar DirectQuery. [2][3]

D5. Variables y filtros

En Grafana, una variable funciona como un filtro que se usa dentro de la consulta SQL. En Power BI, un segmentador filtra los datos del informe. En SolarBI, la variable Dispositivo toma la clave y el nombre desde dwh.dim_dispositivo. Los paneles usan dispositivo_key = ${dispositivo:raw}; por eso, al escoger un inversor, Grafana muestra solamente sus datos. [4]

Bloque 2. Gobernanza de datos

G1. Gobernanza, gestión y calidad

La gobernanza de datos define quién toma decisiones y qué reglas se deben cumplir. La gestión se encarga de guardar, proteger y usar los datos. La calidad revisa que los datos estén completos, no estén repetidos y tengan valores válidos. En SolarBI definí estas reglas y el proceso ETL las aplicó antes de cargar la capa Silver. [5]

G2. Responsables de los datos

El data owner decide para qué se usan los datos y cuáles reglas deben cumplir. El data steward revisa las definiciones y la calidad. El data custodian administra la base de datos, los accesos y los respaldos. Como este trabajo es individual, yo cumplí los tres roles, aunque cada uno tiene una responsabilidad diferente.

G3. Acuerdo sobre los datos

Un contrato de datos deja por escrito qué campos llegan, qué tipo tienen, qué unidad usan y cada cuánto se reciben. También indica qué hacer con datos faltantes, repetidos o fuera de rango. Por ejemplo, si alguien cambia el nombre p_ac_kw sin avisar, el ETL puede fallar y los tableros pueden quedar incompletos. Por eso, cualquier cambio debe probarse y comunicarse antes de usarlo.

G4. Una sola fuente de datos

Una fuente única de verdad significa que todos usan los mismos datos aprobados para hacer los cálculos. El catálogo explica qué significa cada indicador, cómo se calcula, qué unidad usa y de dónde sale. Si Power BI y Grafana usan fórmulas o filtros diferentes, pueden mostrar resultados distintos. Por eso ambos tableros parten de la misma base y de las mismas definiciones.

KPI	Fórmula	Unidad	Fuente	Rol	Día
Yield	Energía / 5 kWp	kWh/kWp	fact_energia_dia	Analista BI	Diaria
Datos válidos	válidas / totales × 100	%	fact_energia_dia	Calidad	Diaria

G5. Protección de datos personales

Las lecturas del inversor no permiten identificar a una persona, así que los datos actuales no son personales. Esto cambiaría si se guardara el nombre del operador que atendió una alarma. En ese caso, solo se debería recoger la información necesaria, limitar el acceso y guardar un registro de quién la consulta. Para los análisis usaría un código interno y dejaría los nombres visibles solo para usuarios autorizados. [6]

Bloque 3. Automatización del proceso ETL

E1. ETL, ELT y Power Query

ETL limpia y transforma los datos antes de guardarlos en su destino. ELT primero los guarda y después los transforma. En SolarBI, Bronze conserva el archivo original, Silver guarda las lecturas válidas y Gold resume la energía y la calidad por día. Usé ETL porque Python limpia los datos antes de cargar Silver y Gold. Power Query también puede hacer este trabajo, pero aquí se usó poco porque la limpieza ya estaba hecha en PostgreSQL.

E2. Carga incremental e idempotencia

Una carga completa vuelve a procesar todos los datos. Una carga incremental trabaja solo con los registros nuevos o modificados. La idempotencia significa que puedo repetir el proceso sin crear duplicados. En SolarBI usé claves primarias y UPSERT para actualizar los registros que ya existían. Después de dos ejecuciones se mantuvieron 832 lecturas en Silver y tres filas diarias en Gold.

E3. Formas de automatizar el proceso

Para este trabajo basta con cron o el Programador de tareas, porque solo se ejecuta un proceso. Airflow sería útil si hubiera muchos pasos, dependencias y reintentos. La actualización programada de Power BI solo renueva el informe publicado; no reemplaza la limpieza ni la carga de PostgreSQL. [7]

Opción	Qué automatiza	Cuándo conviene
cron / Programador	Un proceso programado	Flujo pequeño y estable
Apache Airflow	Varios pasos y reintentos	Procesos más grandes
Power BI refresh	Informe publicado	Datos ya preparados para BI

E4. Actualización incremental en Power BI

RangeStart y RangeEnd indican desde qué fecha y hasta qué fecha se deben actualizar los datos. Power BI puede guardar la historia y renovar solo el periodo más reciente. Esto es útil cuando hay años de información o la actualización tarda mucho. En SolarBI solo hay tres días y tres filas Gold, así que es más fácil actualizar todo. [8]

Bloque 4. Internet de las Cosas

I1. Recorrido de los datos

El inversor mide potencia, irradiancia y temperatura. Un gateway recibe las lecturas y las envía por la red. Bronze guarda los datos originales, Silver guarda los datos válidos y Gold crea el resumen diario. Grafana consulta Silver y Gold, mientras que Power BI usa el resumen Gold. El recorrido es: sensor → gateway → red → Bronze → Silver → Gold → tablero. Así se puede volver desde un resultado hasta la lectura original.

I2. MQTT y entrega de mensajes

MQTT permite que un dispositivo publique mensajes y que otros sistemas los reciban por medio de un broker. Cada mensaje se envía a un topic. QoS 0 no confirma la entrega, QoS 1 puede repetir un mensaje y QoS 2 evita repeticiones, pero usa más comunicación. Para este proyecto usaría QoS 1 y eliminaría los duplicados en Silver. Un topic posible es pascualbravo/solar/sede1/inversor1/p_ac_kw. [9]

I3. Protocolos industriales

En una planta solar, Modbus puede leer los datos de inversores y medidores. Después, un gateway puede enviarlos por MQTT. OPC UA sirve cuando se necesita conectar equipos industriales con información más organizada y con mejores opciones de seguridad.

Protocolo	Modelo	Uso típico	Datos	Seguridad
Modbus RTU/TCP	Comunicación entre equipos	Inversores y medidores	Registros numéricos	Depende de la seguridad de la red
MQTT	Publish/subscribe	Lecturas enviadas al broker	Mensaje configurable	Cifrado, usuario y permisos
OPC UA	Cliente/servidor y pub/sub	Conexión entre equipos	Datos organizados	Cifrado y certificados

I4. Datos por lotes y en tiempo real

El procesamiento por lotes reúne varias lecturas y las trabaja juntas. Sirve para hacer el cierre diario, pero no muestra los cambios al instante. El procesamiento en tiempo real envía los datos casi de inmediato, aunque necesita más control. Al resumir un día, 288 lecturas pasan a una sola fila. Esto facilita el cálculo del rendimiento y el ahorro, pero oculta fallas cortas. Por eso dejé las lecturas de cinco minutos para Grafana y el resumen diario para Power BI.

Bloque 5. Control de versiones

V1. Git, GitHub y archivos del proyecto

Git guarda el historial de cambios en el computador y GitHub publica el repositorio en internet. Un commit registra un cambio y una rama permite trabajar sin afectar la versión principal. Un pull request sirve para revisar antes de unir cambios. El archivo PBIX de Power BI es binario, por lo que Git no puede mostrar fácilmente qué cambió dentro. PBIP separa el proyecto en archivos de texto y facilita la revisión. El tablero de Grafana se guardó como JSON. En esta entrega conservé el PBIX y el JSON. [10]

Parte B. Práctica guiada

Ejecuté todo el proceso desde el archivo etl/run_etl.py. Grafana y Power BI quedaron conectados a la misma base solarbi en Neon. Las credenciales se configuraron solo en mi computador y no se guardaron en GitHub.

Paso 1. Capa Bronze

El simulador creó tres días de lecturas, una cada cinco minutos. También agregó datos faltantes, repetidos y fuera de rango para probar la limpieza. Bronze guardó las 884 filas originales sin hacer cambios.

Paso 1 · Bronze: telemetría sin modificar

ts | dispositivo_id | p_ac_kw | irradiancia_wm2 | temp_modulo_c

2026-10-05 00:00:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:05:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:10:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:15:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:20:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:25:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:30:00 | 1 | 0.0 | 0.0 | 22.0

2026-10-05 00:35:00 | 1 | 0.0 | 0.0 | 22.0

Figura 1. Primeras lecturas guardadas en Bronze.

Paso 2. Limpieza y capa Silver

La limpieza revisó que los campos obligatorios tuvieran datos, que no hubiera lecturas repetidas y que los valores estuvieran dentro de los rangos permitidos. De las 884 filas, 832 fueron válidas. Esto equivale al 94,12 %.

Resultado	Cantidad
Filas leídas	884
Rechazadas por faltantes	17
Rechazadas por duplicidad	20
Rechazadas por rango	15
Filas válidas	832
Porcentaje válido global	94,12 %



Figura 2. Resultado de la limpieza de datos.

Paso 3. Capa Gold e idempotencia

Usé claves primarias y UPSERT para evitar duplicados. Ejecuté el proceso dos veces y el resultado no cambió: quedaron 832 lecturas en silver.lectura_5min y tres resúmenes en dwh.fact_energia_dia. La expresión cron 0 0 * * * permite ejecutarlo todos los días a medianoche.

Tabla	Primera ejecución	Segunda ejecución
silver.lectura_5min	832 filas	832 filas
dwh.fact_energia_dia	3 filas	3 filas

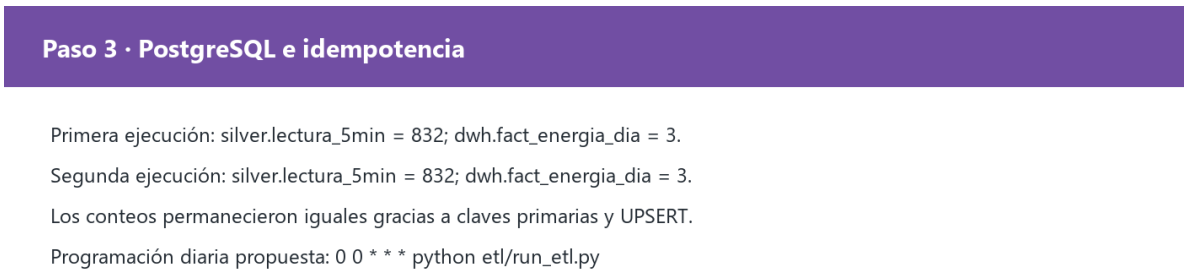


Figura 3. Resultado de dos ejecuciones seguidas.

Paso 4. Informe en Power BI

Power BI se conectó a PostgreSQL usando Import. El informe muestra la energía generada, el rendimiento y el ahorro estimado. El usuario puede escoger una tarifa entre 700 y 1.000 COP/kWh. Para la captura seleccioné 800 COP/kWh como un valor de ejemplo; no significa que sea una tarifa oficial.

Medida	Definición
Energía total (kWh)	SUM(dwh_fact_energia_dia[energia_kwh])
Yield (kWh/kWp)	Energía total / potencia nominal
Tarifa COP	SELECTEDVALUE(ParametroTarifa[Value], 800)
Ahorro estimado	Energía total × Tarifa COP

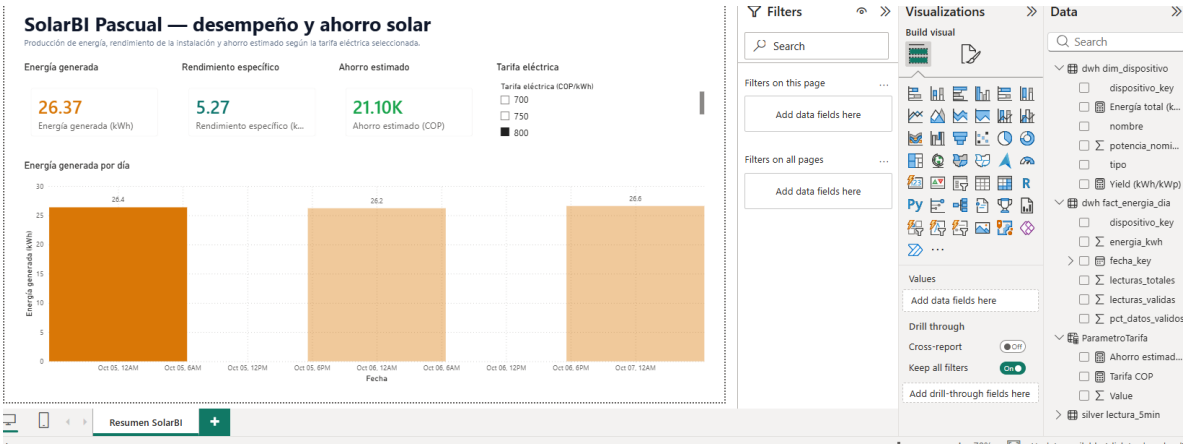


Figura 4. Informe SolarBI Pascual en Power BI.

Paso 5. Dashboard en Grafana

Grafana toma las lecturas de silver.lectura_5min. El filtro $__timeFilter(ts) limita las fechas y la variable Dispositivo permite escoger el inversor. El indicador de calidad usa dwh.fact_energia_dia y muestra 95,56 % para el dispositivo y el periodo seleccionados. La lista de dispositivos sale de dwh.dim_dispositivo. Grafana y Power BI usan la misma base solarbi en Neon.

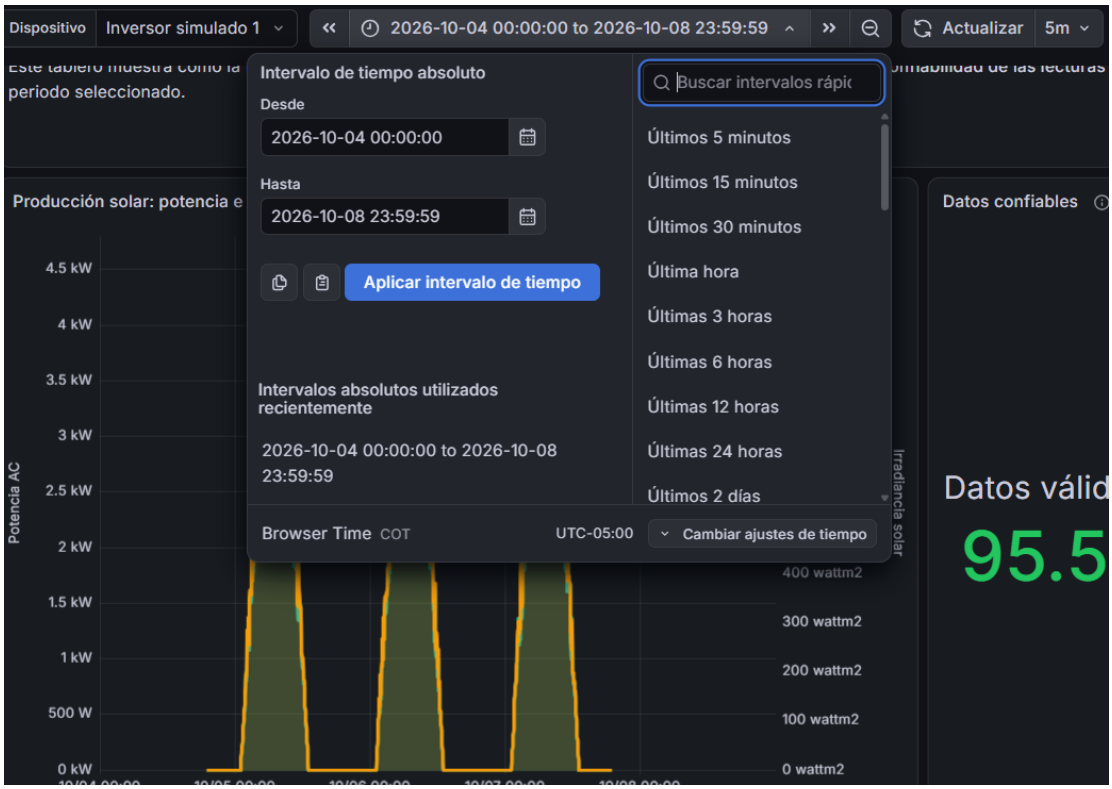


Figura 5a. Dispositivo y periodo seleccionados en Grafana.



Figura 5b. Potencia, irradiancia y datos válidos en Grafana.

Paso 6. Reflexión

Grafana me resultó más útil para revisar las lecturas y ver cambios frecuentes. Power BI fue más cómodo para mostrar los cálculos de energía, rendimiento y ahorro. La parte más difícil fue lograr que las dos herramientas usaran la misma conexión, los mismos tipos de datos y los mismos filtros. Usaría Grafana para el seguimiento diario de un operador y Power BI para presentar resultados. Mantendría Gold como base de los indicadores y Silver para revisar errores o datos extraños.

Parte C. Repositorio y verificación

Repositorio: https://github.com/savelasquezs/Grafana_power_bi

Elemento	Verificación
Autor	Santiago Velásquez Serna · 1038414022
Modalidad	Trabajo individual
Rama verificada	main
ETL	Se ejecuta con python etl/run_etl.py
Grafana	Dashboard guardado en grafana/dashboard.json
Power BI	powerbi/SolarBI_Pascual.pbix
Seguridad	No se guardaron credenciales en GitHub

El trabajo fue individual, por eso el historial tiene un solo autor. Los commits separan los cambios del ETL, Grafana, Power BI y la entrega final.

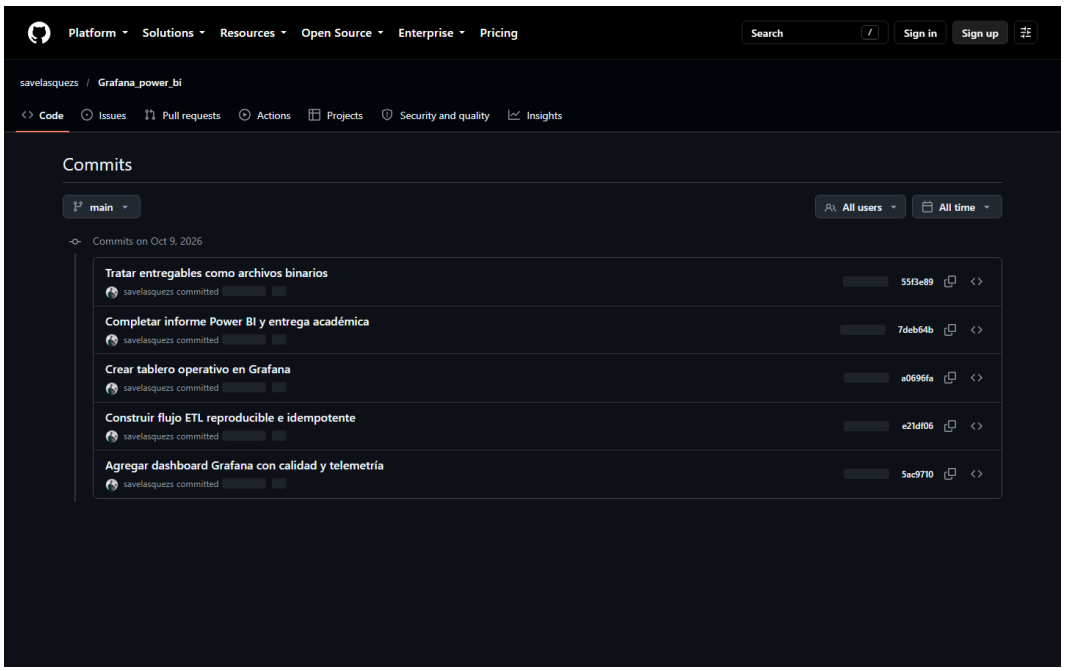


Figura 6. Historial de commits en la rama main.

Referencias

[1] Grafana Labs. Create dashboards y conceptos de data sources. https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/create-dashboard/

[2] Microsoft Learn. DirectQuery in Power BI. https://learn.microsoft.com/power-bi/connect-data/desktop-directquery-about

[3] Microsoft Learn. Data refresh in Power BI. https://learn.microsoft.com/power-bi/connect-data/refresh-data

[4] Grafana Labs. PostgreSQL template variables. https://grafana.com/docs/grafana/latest/datasources/postgres/template-variables/

[5] DAMA International. DAMA-DMBOK. https://www.dama.org/cpages/body-of-knowledge

[6] Colombia. Ley 1581 de 2012, protección de datos personales. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

[7] Apache Airflow. Cron & Time Intervals. https://airflow.apache.org/docs/apache-airflow/stable/authoring-and-scheduling/cron.html

[8] Microsoft Learn. Incremental refresh overview. https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview

[9] OASIS. MQTT Version 5.0. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[10] Microsoft Learn. Power BI Desktop projects (PBIP). https://learn.microsoft.com/power-bi/developer/projects/projects-overview